

– Tutorial – Control de Versions en C

Git i GitHub

Grau d'Enginyeria Informàtica - Introducció a la Programació

Curs 2026-27

Índex

1	Introducció	2
2	Exercici 1: Git Local	3
2.1	Configurant Git	3
2.2	Creant el repositori	3
2.3	El fitxer .gitignore	3
2.4	Sincronitzant (Add i Commit)	4
3	Exercici 2: Git → GitHub	4
3.1	Crear repositori remot	4
3.2	Generar un token per accedir al repositori remot	5
3.3	Vincular i Pujar (Push)	6
4	Exercici 3: Clonar (Aula vs Portàtil)	8
5	Exercici 4: Resolució de Conflictes	8
5.1	Simulació del Conflicte	8

1 Introducció

Quan treballem en un projecte de desenvolupament de codi, és important poder guardar els canvis que realitzem, consultar-ne l'historial i recuperar versions anteriors del projecte. Per fer-ho, utilitzarem el concepte de **repositori** i l'eina de control de versions **Git**.

Un **repositori** és un espai on s'emmagatzema un projecte i la informació necessària per gestionar-lo. En el context del control de versions, el repositori també permet conservar l'historial dels canvis que s'han realitzat sobre el projecte.

Git és un sistema de control de versions que permet gestionar els canvis d'un projecte i mantenir-ne un historial. Git treballa amb repositoris, on s'emmagatzema la informació necessària per controlar les diferents versions del projecte. Cada vegada que volem registrar un conjunt de canvis, podem crear un *commit*. Un *commit* representa, per tant, un punt concret de l'historial del projecte que podem consultar posteriorment.

En treballar amb Git, és habitual tenir un repositori al nostre ordinador i una còpia del repositori en un servidor extern. Per això, distingim entre:

1. **Repositori local:** és el repositori Git que es troba al nostre ordinador. És on treballem habitualment i on Git emmagatzema l'historial de *commits* del projecte.
2. **Repositori remot:** és un repositori Git que es troba en un servidor extern. Permet compartir el projecte amb altres persones i disposar d'una còpia del repositori fora del nostre ordinador. GitHub i GitLab son algunas de les moltes plataformes web que ofereixen hosting per a repositoris remots de Git. Nosaltres usarem GitHub.

La Figura 1 mostra les diferents zones que intervenen en el flux de treball habitual amb Git. El procés comença a l'**espai de treball**, on modifiquem els fitxers del projecte.

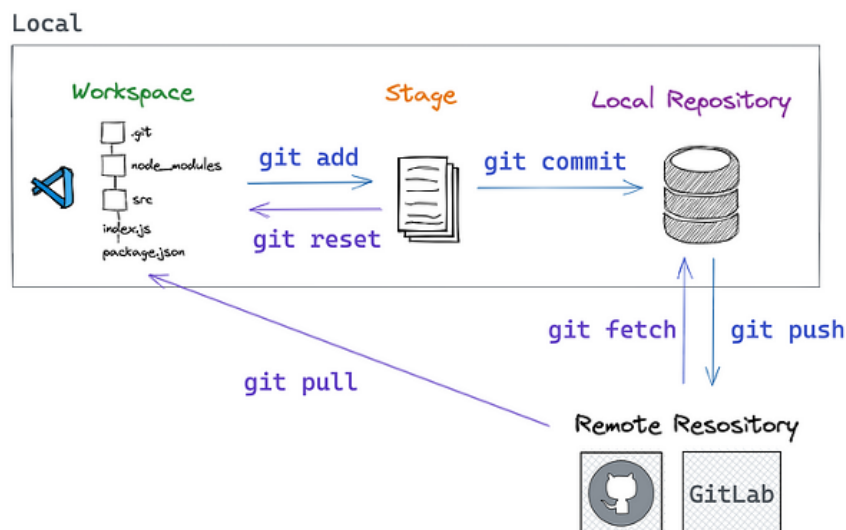


Figura 1: Flux de treball bàsic de Git entre l'espai de treball, l'àrea de preparació, el repositori local i el repositori remot.

Quan volem seleccionar quins canvis formaran part del pròxim *commit*, els afegim a l'àrea de preparació (*Stage*) mitjançant l'ordre `git add`.

A continuació, l'ordre `git commit` registra aquests canvis en l'**repositori local**. A partir d'aquest moment, els canvis formen part de l'historial de Git.

Finalment, mitjançant l'ordre `git push`, podem enviar els *commits* del repositori local al **repositori remot**. D'aquesta manera, el projecte queda disponible en un servidor extern i pot ser compartit amb altres desenvolupadors.

Els exercicis següents ens permetran posar en pràctica aquest flux de treball.

2 Exercici 1: Git Local

2.1 Configurant Git

Si és el primer cop que uses Git a l'ordinador, configura la teva identitat (posa el teu nom i mail però sense les `<>`):

```
git config --global user.name <El teu nom>
git config --global user.email <el_teu_email@ub.edu>
```

⚠️ Avís: Si copieu i enganxeu els codis, és possible que s'hi incloguin espais que no haurien d'aparèixer i que, en conseqüència, el codi no funcioni correctament. Per tant, **cada vegada que executeu un codi copiat d'aquest PDF, comproveu que sigui exactament igual que el que apareix al document.**

2.2 Creant el repositori

Crearem una carpeta per al projecte i la inicialitzarem. Obre la terminal (CMD o PowerShell):

```
mkdir Prog-C-Git-Test
cd Prog-C-Git-Test
git init
```

La comanda `git init` crea una carpeta oculta `.git` on es guardarà l'històric.

2.3 El fitxer `.gitignore`

A continuació, indicarem a Git que ignori determinats fitxers, per exemple, els fitxers executables, com ara els fitxers `.exe` a **Windows**, o els fitxers objecte (`.o`) generats pel compilador `gcc`. L'objectiu és pujar a GitHub únicament els fitxers essencials necessaris per executar el projecte, com ara els fitxers de codi (`.c` y `.h`) i de configuració, i ignorar els fitxers generats durant el procés de compilació.

Recorda que pots utilitzar l'ordre `ls -l` des de la terminal per comprovar les extensions dels fitxers.

1. Crea un fitxer anomenat `.gitignore` (el punt inicial és important) amb aquest contingut. Una vegada creat, com que es ocult (comença amb `.`) per a veure'l amb `ls` has de fer `ls -al`:

```
# Ignorar executables i binaris
*.exe
*.o
*.out

# Ignorar fitxers de configuració d'editors (brossa)
```

```
.vscode/  
.idea/  
.DS_Store
```

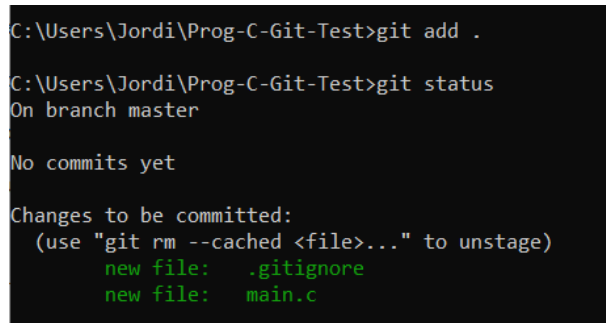
2. Crea el fitxer del codi font `main.c`:

```
#include <stdio.h>  
  
int main(void) {  
    printf("Hola, Món!\n")  
    return 0;  
}
```

2.4 Sincronitzant (Add i Commit)

Ara tenim fitxers creats, però Git encara no "els vigila".

1. `git add .`: Afegeix tots els fitxers (respectant el `.gitignore`) a la zona d'espera (staging area).
2. `git status`: Comprova què s'està apunt de guardar.



```
C:\Users\Jordi\Prog-C-Git-Test>git add .  
  
C:\Users\Jordi\Prog-C-Git-Test>git status  
On branch master  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   .gitignore  
    new file:   main.c
```

Figura 2: Exemple de sortida de `git status`. Fixeu-vos que només s'afegeixen els fitxers font i no els executables. Nota: aquesta captura de pantalla s'ha fet des de Windows. Si utilitzeu Linux o macOS, les barres de les rutes es mostren invertides: `/` en lloc de `\`.

Confirmem els canvis:

```
git commit -m "Primer commit: Hello World en C i gitignore"
```

3 Exercici 2: Git → GitHub

3.1 Crear repositori remot

1. Accedeix a github.com, inicia sessió i prem el botó **New Repository**.
2. Anomena'l `Prog-C-Git-Test`.
3. **⚠ Important:** Deixa **desmarcades** les opcions “Add README” (Off), “Add .gitignore” (No .gitignore) i “License” (No licence). Volem un repositori buit (il·lustrat a la Figura 3).

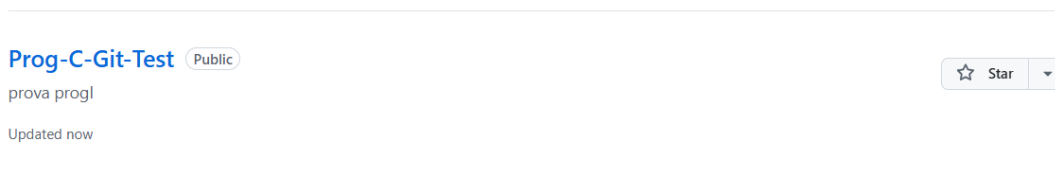


Figura 3: Configuració correcta del nou repositori. Ha d'estar buit (sense README).

3.2 Generar un token per accedir al repositori remot

Per sincronitzar el repositori local amb el remot, cal primer generar un *token* que permetrà iniciar sessió al servidor remot mitjançant el terminal, tal com es detalla a continuació.

1. En el teu perfil de GitHub (part superior dreta) accedir a Settings → Developer settings → Personal access tokens → Tokens (classic); il·lustrat a la Figura 4.
2. A continuació, seleccioneu Generate new token (classic).
3. Quan se us demani (tal com s'il·lustra a la Figura 5), introduïu una nota descriptiva per identificar l'ús del *token* (per exemple, el repositori associat); establiu una caducitat de 90 dies; seleccioneu el permís (*scope*): ☒ repo i feu clic a *Generate token*.

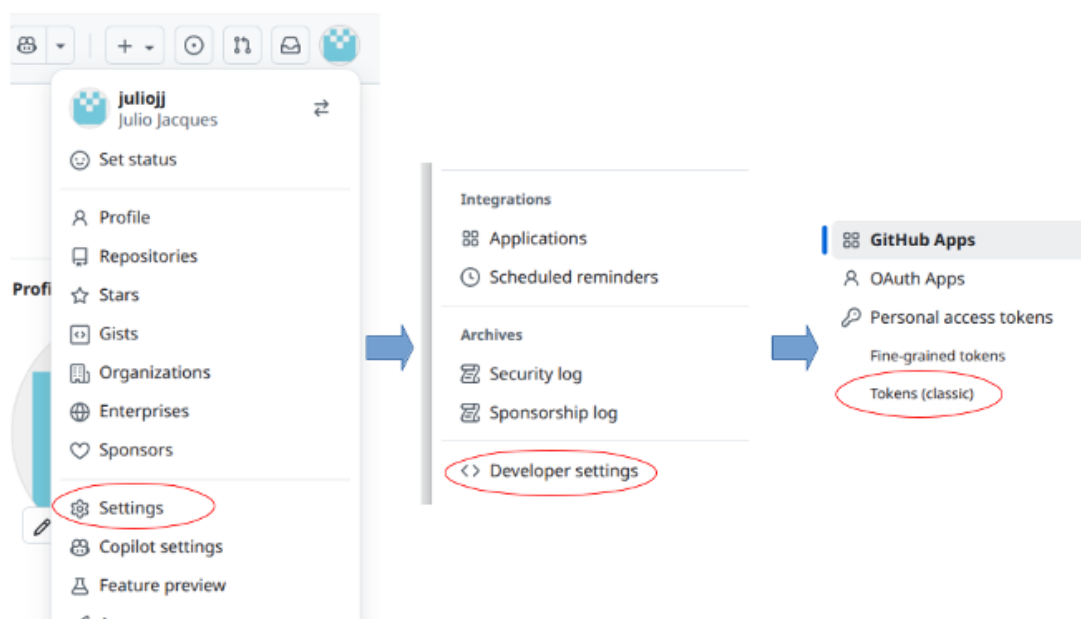


Figura 4: Generació d'un token clàssic per accedir al repositori remot.

⚠ Important: Deseu el *token* generat (una cadena de caràcters força llarga) en un lloc segur, per exemple al vostre correu electrònic. Aquest *token* s'utilitzarà com a contrasenya per accedir al repositori remot en futures sessions de laboratori. Si el perdeu, haureu de generar-ne un de nou.

Note

Prog-C-Git-Test

What's this token for?

Expiration

📅 90 days (Dec 01, 2026) ▾

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☒ workflow	Update GitHub Action workflows

Figura 5: Configuració del token clàssic.

3.3 Vincular i Pujar (Push)

GitHub et mostrarà unes instruccions. Copia les que corresponen a “...or push an existing repository from the command line” i executa-les des de la teva terminal local, com es mostra a continuació. Fixeu-vos que les línies en verd que comencen amb # són comentaris, és a dir, no es processen en cap moment:

```
# Substitueix 'usuari' pel teu nom d'usuari de GitHub
git remote add origin https://github.com/usuari/Prog-C-Git-Test.git
git branch -M main
git push -u origin main
```

Si heu configurat correctament el *token*, apareixerà una finestra que us demanarà que l'introduïu (Figura 6). Un cop hàgiu introduït un *token* vàlid, els fitxers es pujaran al vostre repositori GitHub. Si tot ha anat bé, veuràs un missatge com el que es mostra a la Figura 7. Refrescant la pàgina de GitHub hauries de veure el teu codi `main.c` al repositori Prog-C-Git-Test.

En els *commits* posteriors, només serà necessari executar l'ordre `git push` per sincronitzar els canvis amb el repositori remot prèviament configurat. Això és possible perquè el *branch* local actual i el *branch master* del repositori local ja es troben correctament configurats.

Errorrs comuns i possibles solucions: en cas que aparegui l'error *Invalid username or token* en fer un `git push`, podeu restablir les credencials executant l'ordre següent des de la terminal, cosa que us obligarà a tornar a introduir el *token* generat:

```
echo "protocol=https\nhost=github.com" | git credential-manager erase
git push
```

⚠ Important: Es recomana **NO modificar directament els fitxers mitjançant la interfície web de GitHub**, ja que això pot provocar divergències entre el repositori local i el remot i generar conflictes durant la posterior

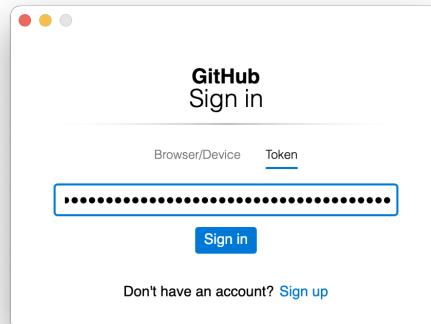


Figura 6: Iniciem sessió amb el token generat.

```
C:\Users\Jordi\Prog-C-Git-Test>git remote add origin https://github.com/ElRevanchista/Prog-C-Git-Test.git
C:\Users\Jordi\Prog-C-Git-Test>git branch -M main
C:\Users\Jordi\Prog-C-Git-Test>git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 435 bytes | 87.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ElRevanchista/Prog-C-Git-Test.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
C:\Users\Jordi\Prog-C-Git-Test>
```

Figura 7: Resultat d'un `git push` correcte. Fixeu-vos que s'han pujat 4 objectes al servidor (el fitxer `.c`, el fitxer `.gitignore` i alguns fitxers de configuració). Nota: aquesta captura de pantalla s'ha fet des de Windows. Si utilitzes Linux o macOS, les barres de les rutes es mostren invertides: `/` en lloc de `\`.

sincronització dels canvis. Per aquest motiu, es recomana realitzar les modificacions localment, confirmar-les mitjançant un *commit* i, finalment, executar `git push`. Més endavant, veurem com descarregar el projecte de GitHub al vostre ordinador local mitjançant l'ordre `git clone`.

4 Exercici 3: Clonar (Aula vs Portàtil)

Quan canviis d'ordinador, no copiis els fitxers en un pendrive. Utilitza Git. És a dir, clonar el vostre projecte des del vostre repositori de Git.

```
git clone https://github.com/el_teu_usuari/Prog-C-Git-Test.git
cd Prog-C-Git-Test
```

Tingueu en compte que, si heu seguit les instruccions anteriors, no tindreu els fitxers executables al vostre repositori de Git (recordeu el `.gitignore`). Per tant, haureu de tornar a compilar el codi per poder executar-lo.

5 Exercici 4: Resolució de Conflictes

Un conflicte es produeix, per exemple, quan dues persones modifiquen la mateixa línia d'un fitxer i Git no pot determinar quina versió ha de conservar, o quan intenteu fer un `git push` d'una nova versió després d'haver fet modificacions al repositori de Git mitjançant la interfície web (consulteu la nota al final de la Secció 3.3 per evitar aquest problema).

5.1 Simulació del Conflicte

Primer, cloneu el vostre repositori de Git, amb `git clone`.

Pas 1 (Canvi Remot): Ves a la web de GitHub, edita `main.c` i canvia el missatge del `printf` per "Editat des de GitHub". Fes "Commit changes" a la web.

Pas 2 (Canvi Local): Al teu ordinador, sense fer `git pull`¹, edita `main.c` i canvia el missatge per "Editat des de Local".

Pas 3 (El Xoc): Intenta pujar el teu canvi local (amb `git commit` i `git push`).

```
git add .
git commit -m "Canvi local conflictiu"
git push
```

Git rebutjarà la connexió perquè hi ha canvis al núvol que no tens. Veuràs un missatge d'error vermell com aquest:

¹`git pull` actualitza el vostre repositori local amb els canvis més recents del repositori remot.

```
C:\Users\Jordi\Prog-C-Git-Test>git push
To https://github.com/ElRevanchista/Prog-C-Git-Test.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/ElRevanchista/Prog-C-Git-Test.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

C:\Users\Jordi\Prog-C-Git-Test>_
```

Figura 8: Error de conflicte. Git ens avisa que "remote contains work that you do not have locally".

Pas 4 (git pull): La comanda git pull intentarà sincronitzar el repositori local amb el repositori remot però falla.

```
git pull
```

Pas 5 (merge): Haureu de resoldre el conflicte manualment. La comanda següent intentarà fusionar els dos fitxers.

```
git config pull.rebase false # merge
```

Obre main.c i resol el conflicte manualment esborrant les marques «<, ==, >>», i conservant el printf que considereu correcte. A continuació, executeu les comandes següents.

```
# Un cop arreglat el codi C:
git add main.c
git commit -m "Conflicte resolt"
git push
```